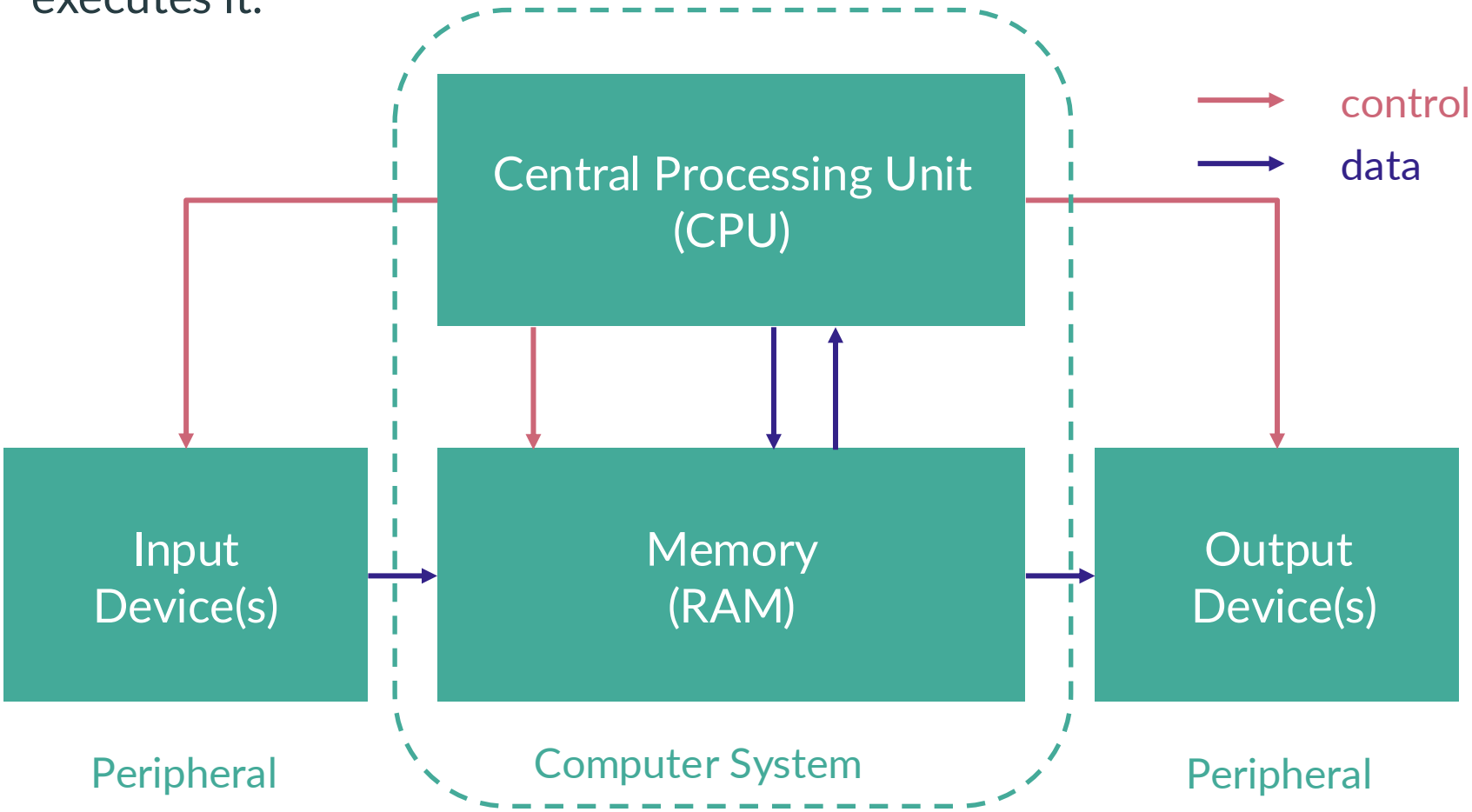
CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

CPU Structure

Central Processing Unit

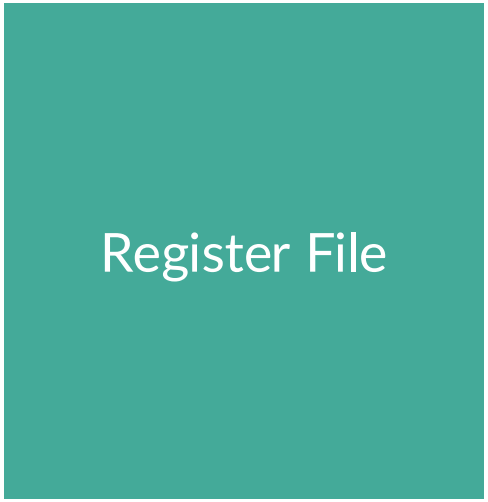
The CPU fetches the next instruction in a program from memory and executes it.



(Logical)
CPU Structure

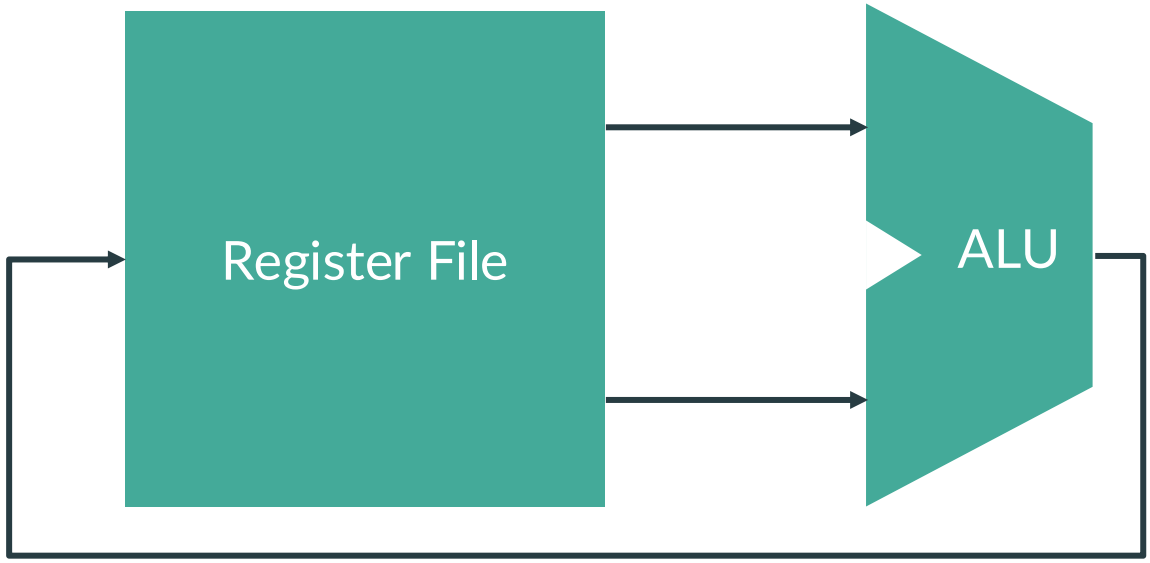
Register File

The *register file* is a small, fast memory unit used to store current data being processed.



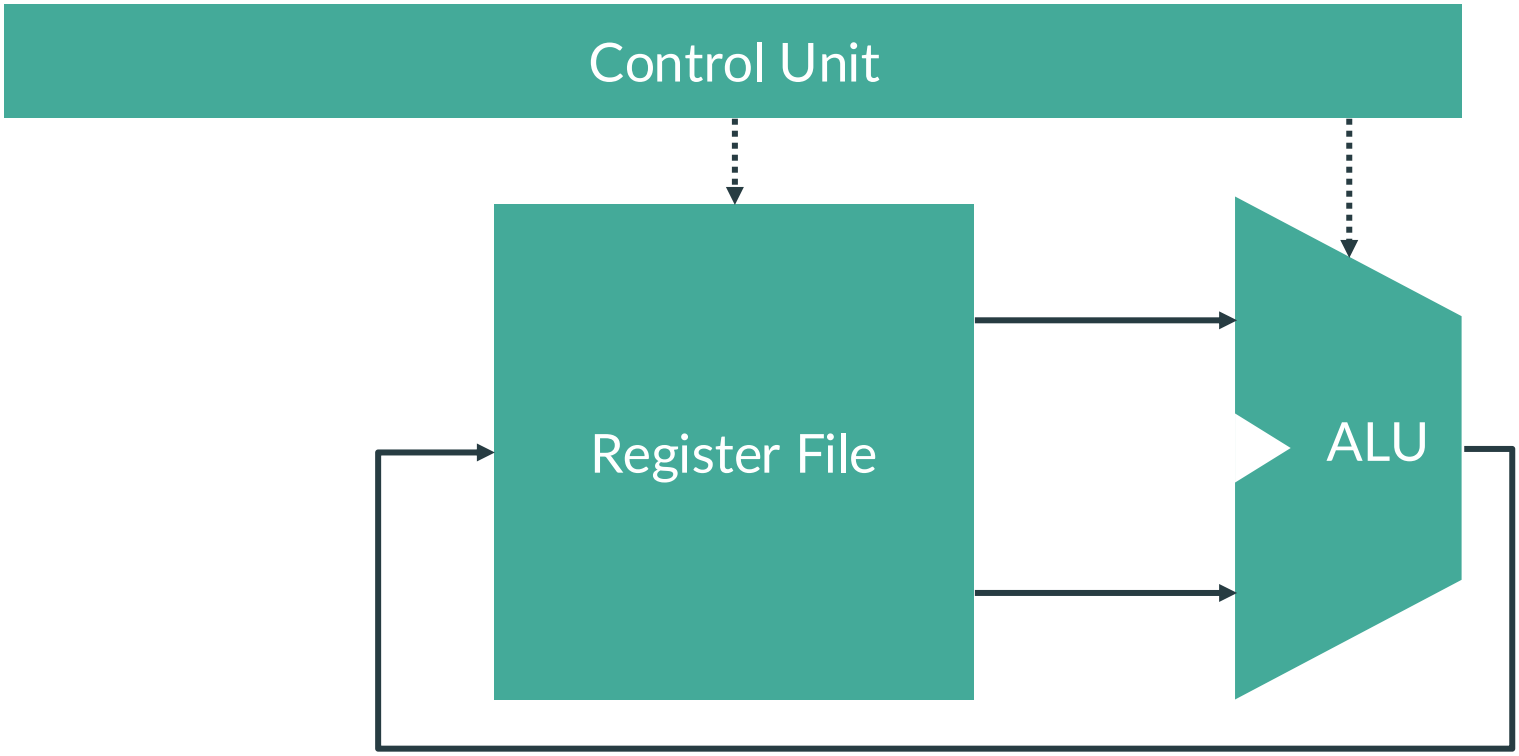
Arithmetic Logic Unit (ALU)

The *arithmetic logic unit (ALU)* performs all mathematic and logical operations.



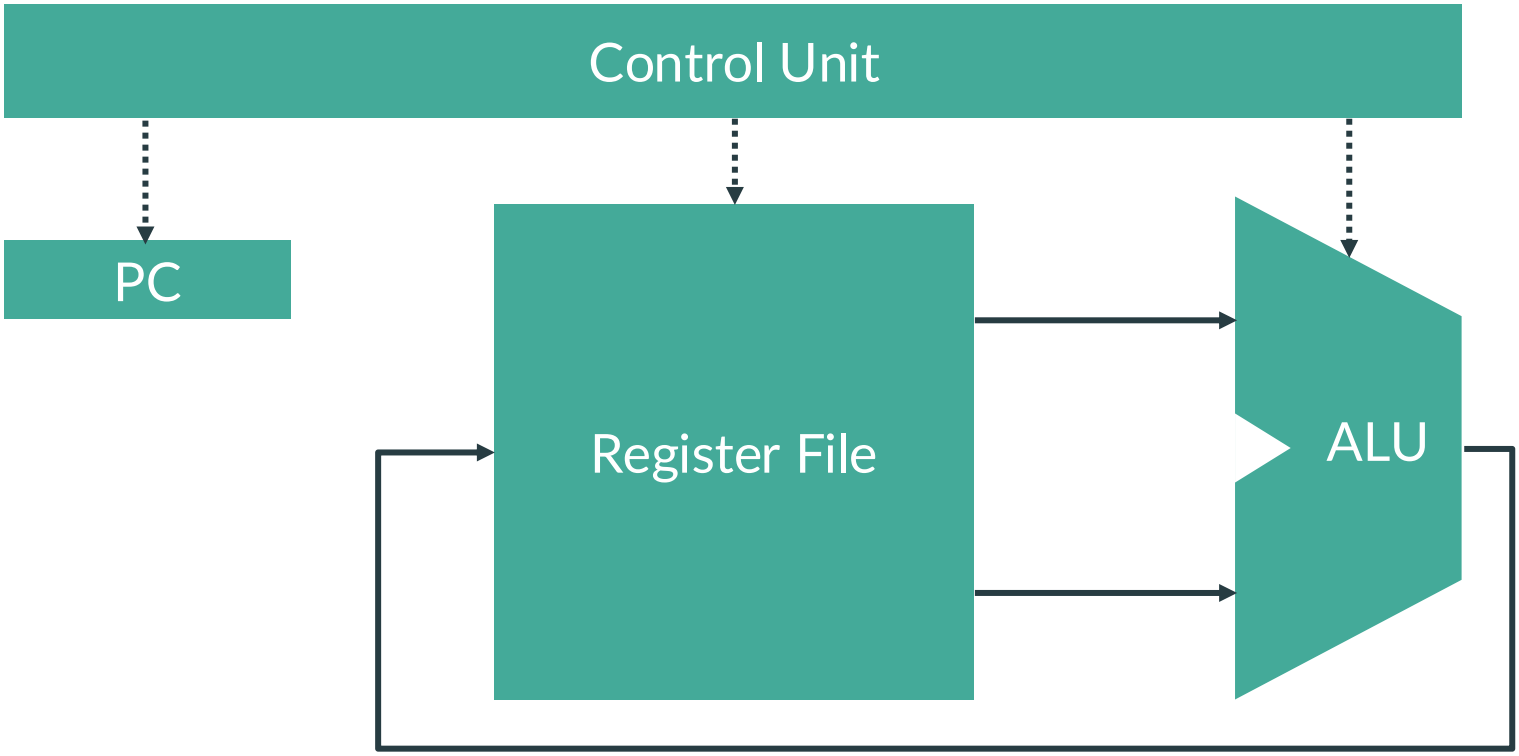
Control Unit

The *control unit* directs the flow of data between the register file, ALU, and memory.



Program Counter (PC)

The *program counter (PC)* keeps track of where the CPU is in the program.

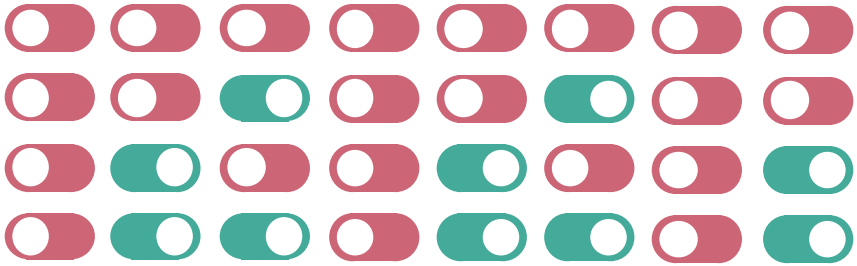


Register File

A Register is a Memory Device

A *register* is a memory device that stores a single word of data (usually 32 or 64 bits).

A 32 bit register contains 32 switches and can represent 2^{32} values:



A Register File Contains Many Registers

A *register file* is a small, fast block of memory inside the CPU that holds a fixed set of registers used for computation.

It is used to store:

1. Input values used in a computation
2. The output value of a computation

A Register File Contains Many Registers

A *register file* is a small, fast block of memory inside the CPU that holds a fixed set of registers used for computation.

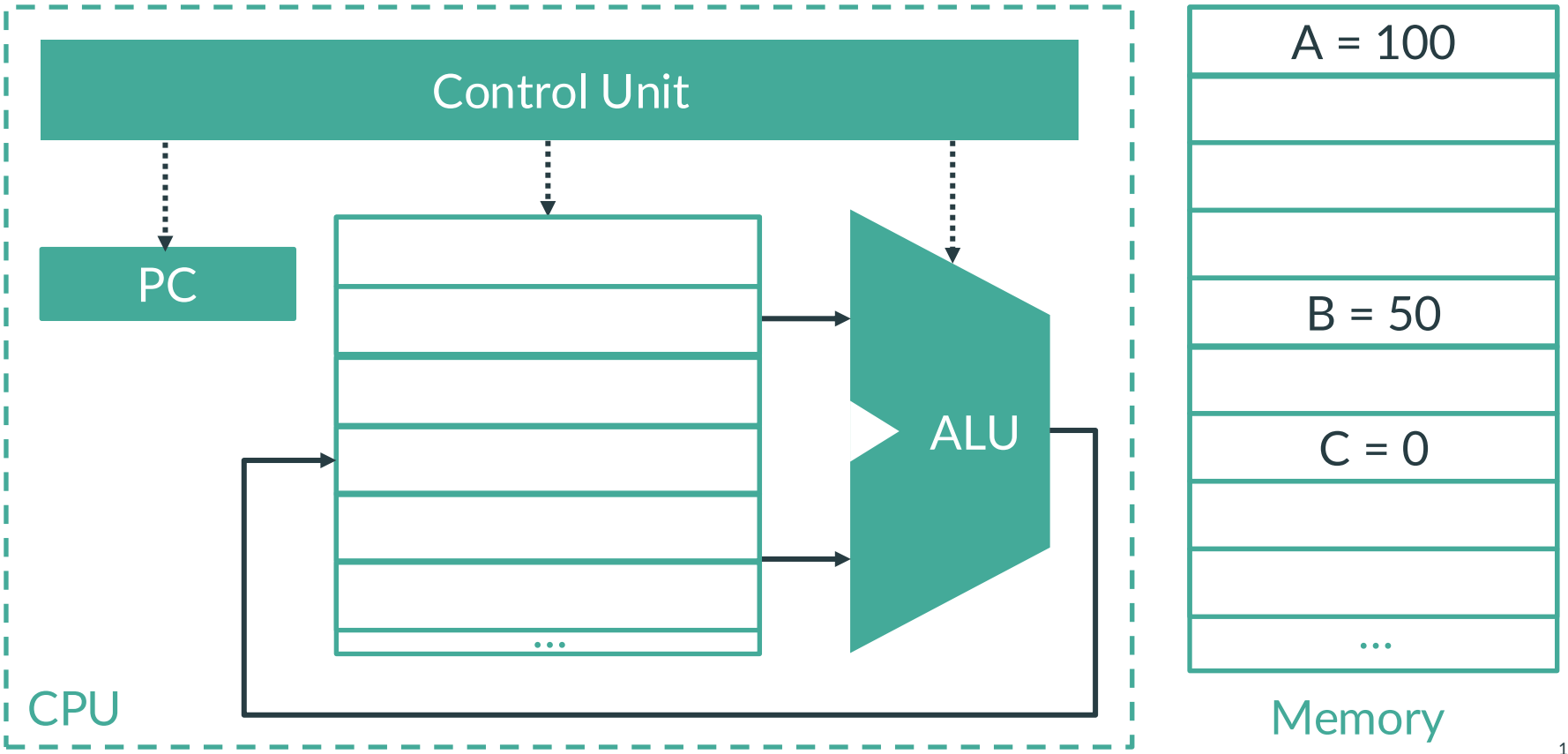
It is used to store:

1. Input values used in a computation
2. The output value of a computation

★ All data must be loaded into a register before using it for computation

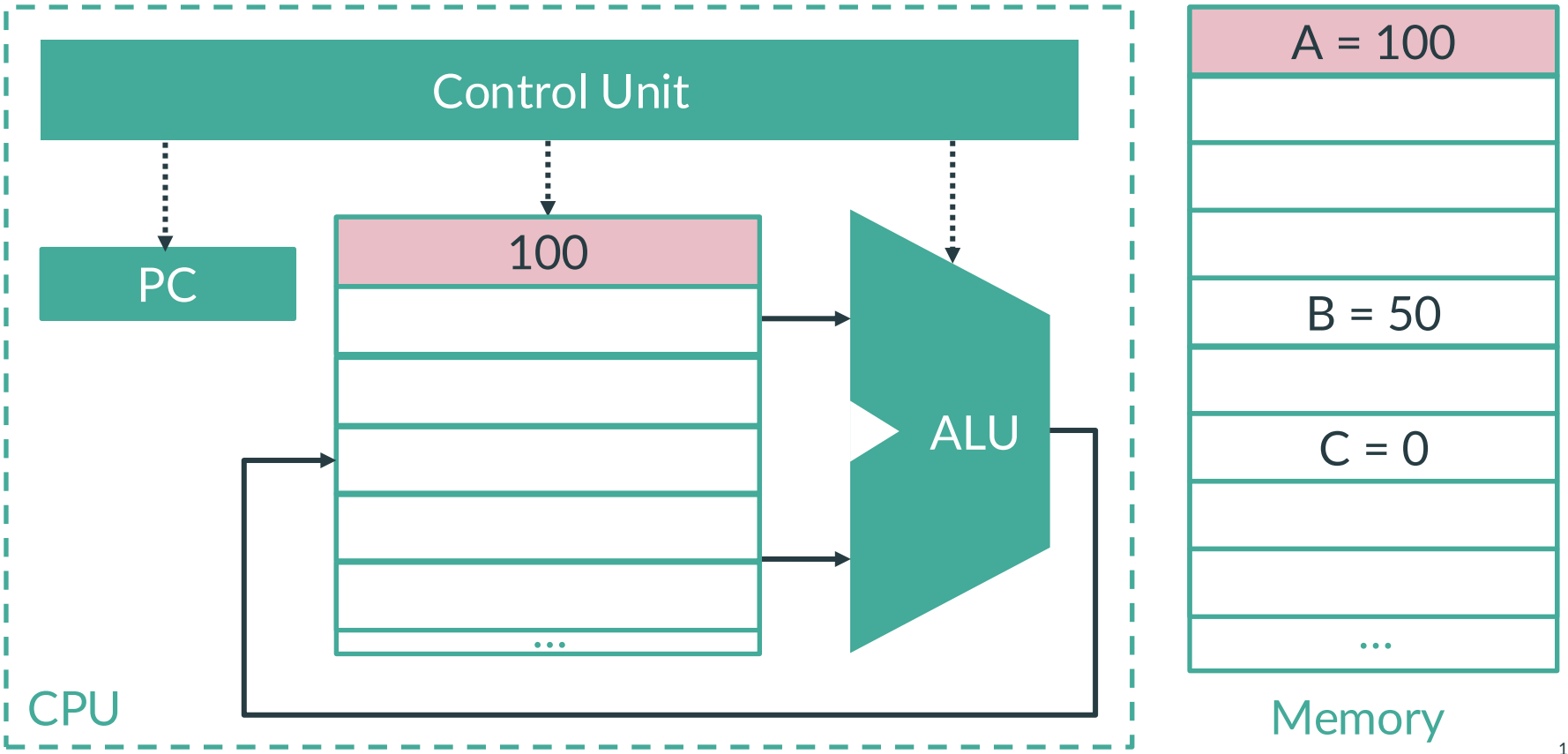
Using the Register File: C = A + B

Goal: update the value of C to be the result of adding A and B.



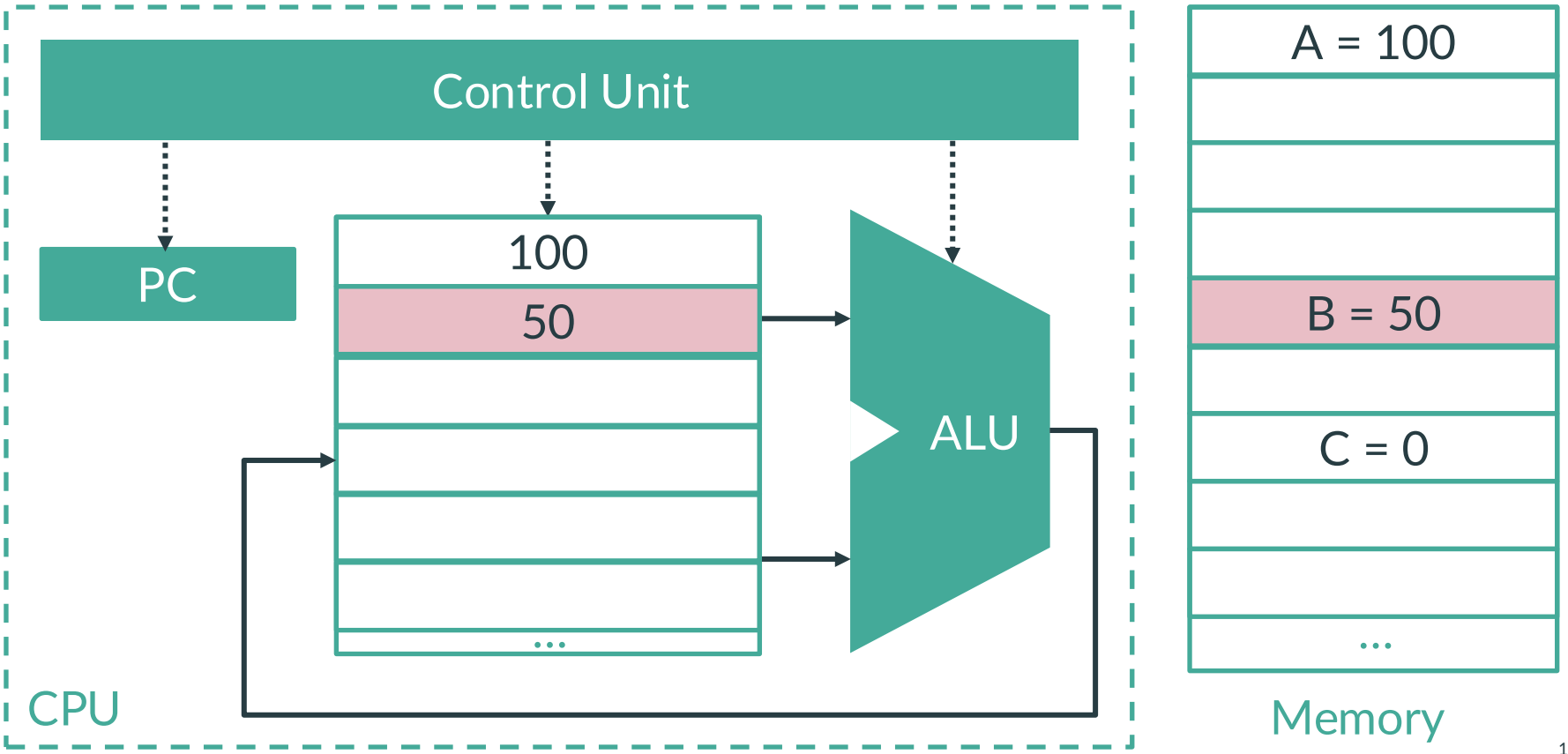
Using the Register File: C = A + B

1. The CPU issues a read command to load the value of A into a register.



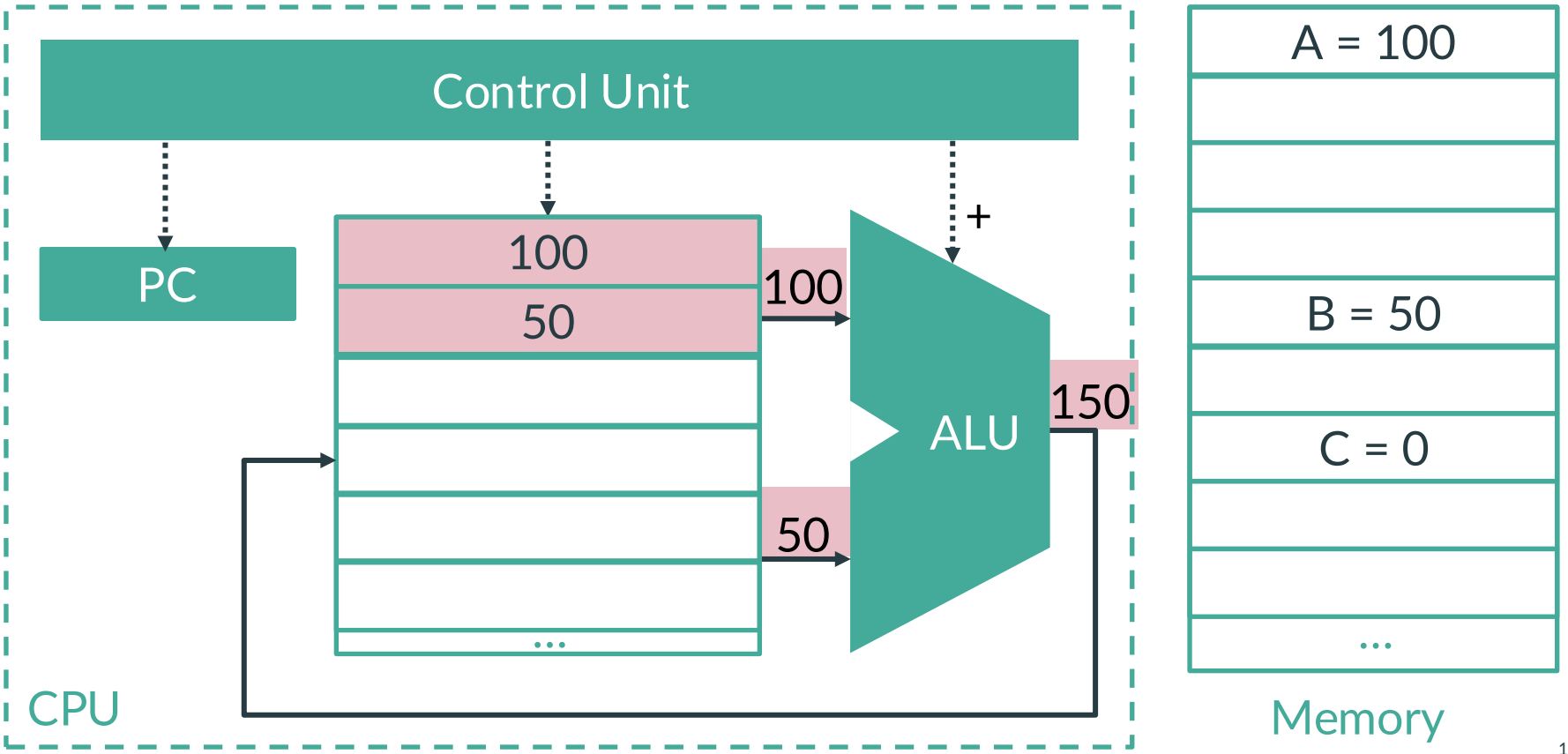
Using the Register File: C = A + B

2. The CPU issues a read command to load the value of B into a different register.



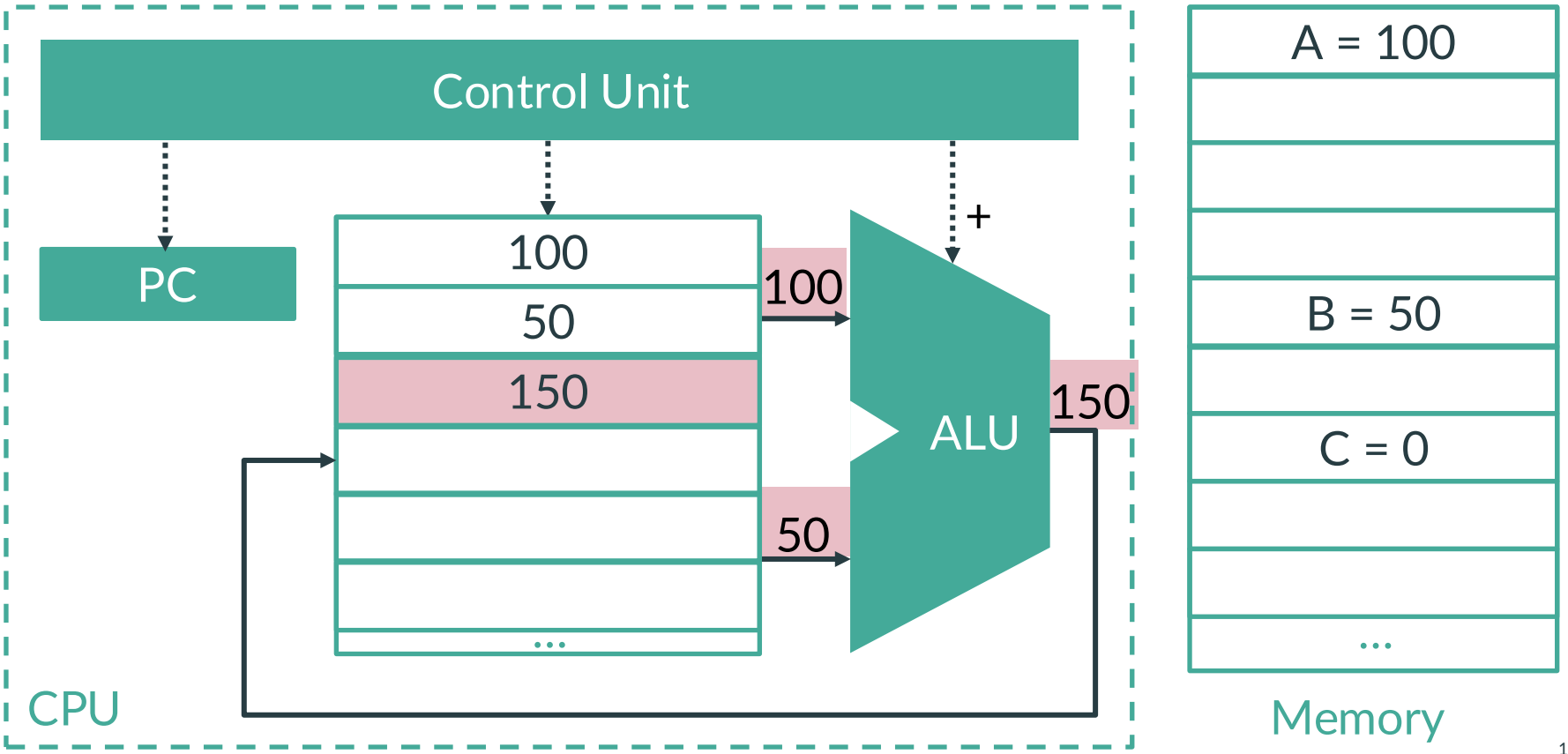
Using the Register File: C = A + B

3. The CPU performs the addition.



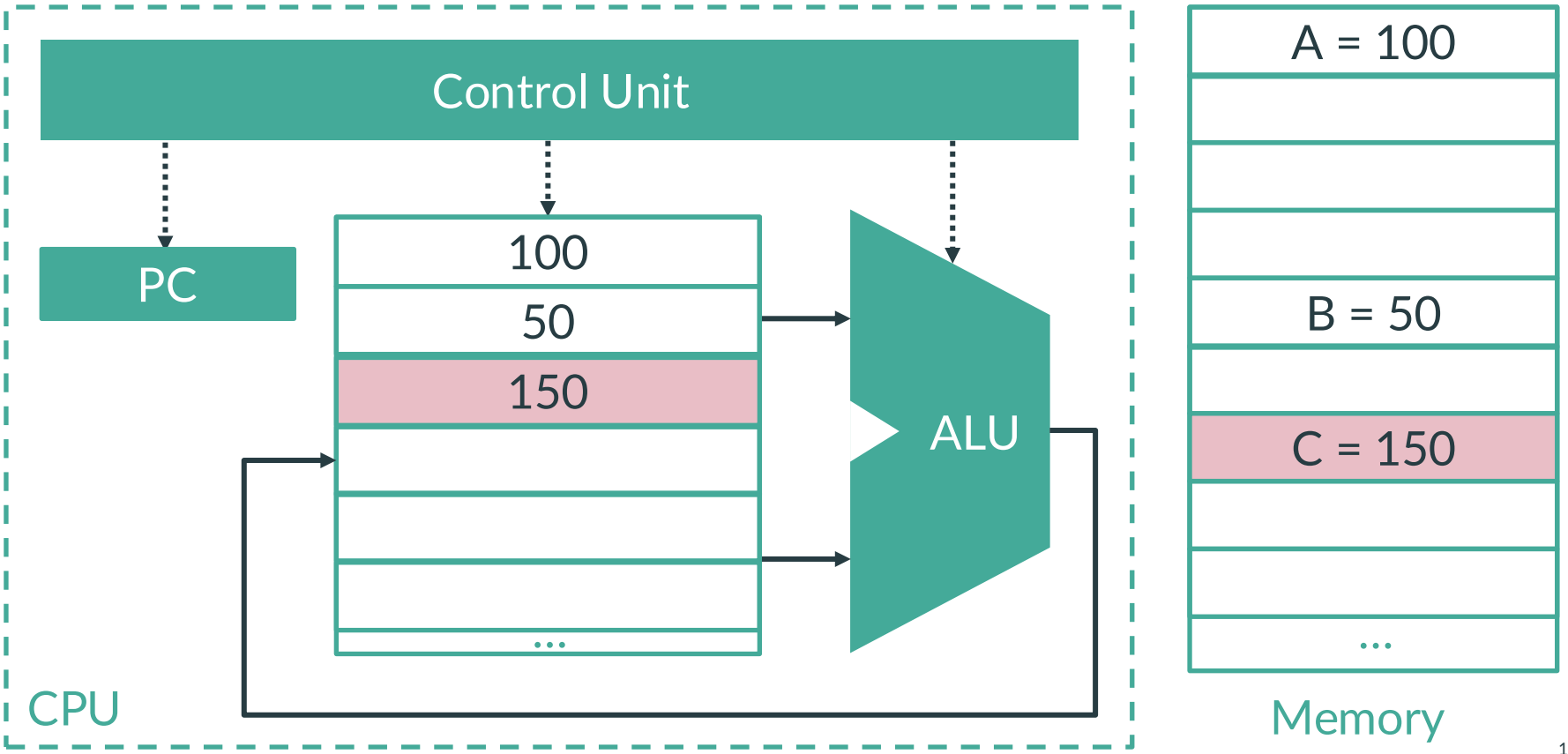
Using the Register File: C = A + B

4. The CPU puts the result of the addition in another register.



Using the Register File: C = A + B

5. The CPU issues a write command to store the result of computation in memory.

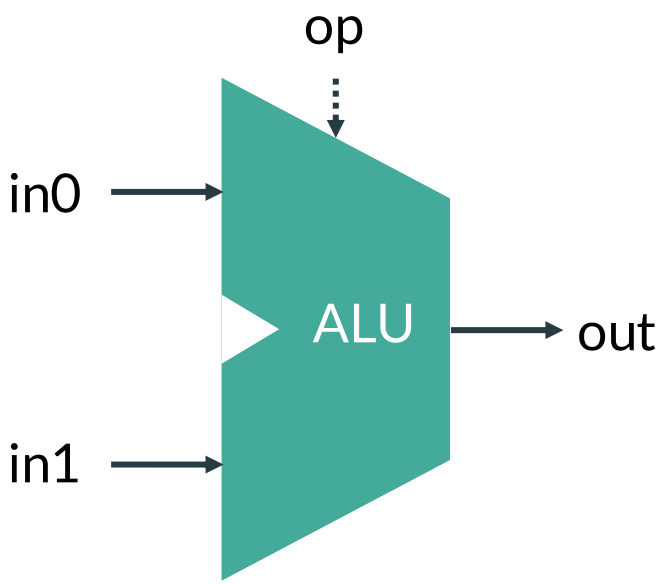


Arithmetic Logic Unit (ALU)

The ALU Performs Computation

The *arithmetic logic unit (ALU)* performs all mathematic and logical operations.

It takes two inputs, performs an operation, and produces one output.



ALU Operations

Operation	Example	Description
Addition	A + B	add two binary values
Subtraction	A - B	subtract two binary values
Set Less Than	A < B	1 if (A<B) else 0
Set Equal To	A == B	1 if (A==B) else 0
And	A & B	bitwise and
Or	A | B	bitwise or
Xor	A ^ B	bitwise xor
Not	~A	bitwise not
Shift Left	A << n	move bits left by n places
Shift Right	A >> n	move bits right by n places

* This is not every possible ALU op, nor does every ALU have all these ops.

Logical Operations

AND:

0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1

OR:

0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1

XOR:

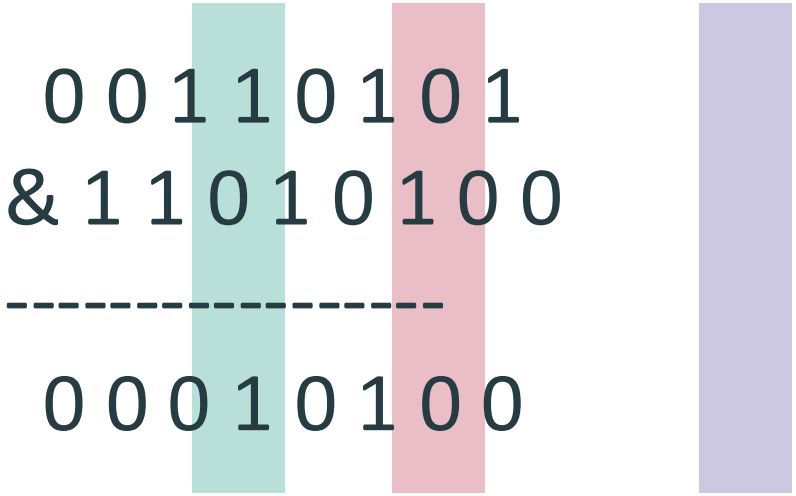
0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0

NOT:

~0 = 1
~1 = 0

Bitwise Logic

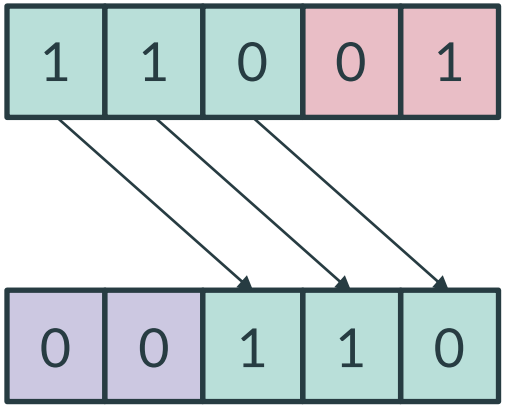
Bitwise logic operations manipulate individual bits of binary data. Each bit in the result depends only on the corresponding bits of the inputs.



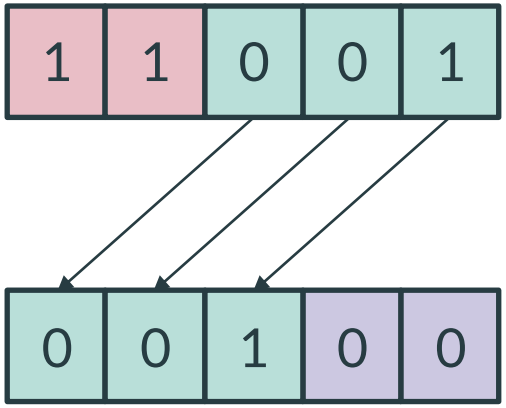
Logical Shift

A logical shift moves all bits in a binary value left or right and fills empty positions with zeros.

11001 >> 2



11001 << 2



Shifters Are Multipliers

Left shift == multiply by 2

00001 << 1 = 00010

1 × 2 = 2

00001 << 2 = 00100

1 × 4 = 4

$A << n = A \times 2^n$

Shifters Are Multipliers

Left shift == multiply by 2

00001 << 1 = 00010

1 × 2 = 2

00001 << 2 = 00100

1 × 4 = 4

$A << n = A \times 2^n$

Right shift == divide by 2

01000 >> 1 = 00100

8 ÷ 2 = 4

01000 >> 2 = 00010

8 ÷ 4 = 2

$A >> n = A \div 2^n$

Bitwise and Shift Operators in C

C provides bitwise operators for AND, OR, XOR, and NOT.

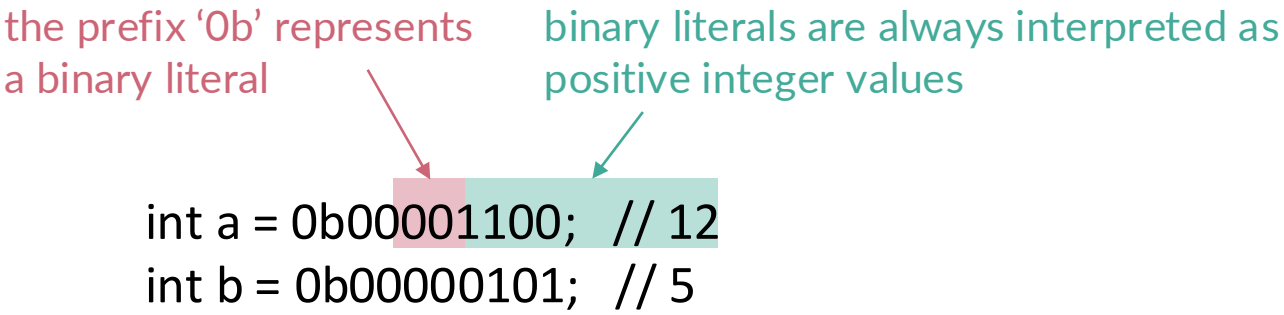
the prefix '0b' represents a binary literal



```
int a = 0b00001100;   // 12
int b = 0b00000101;   // 5
```

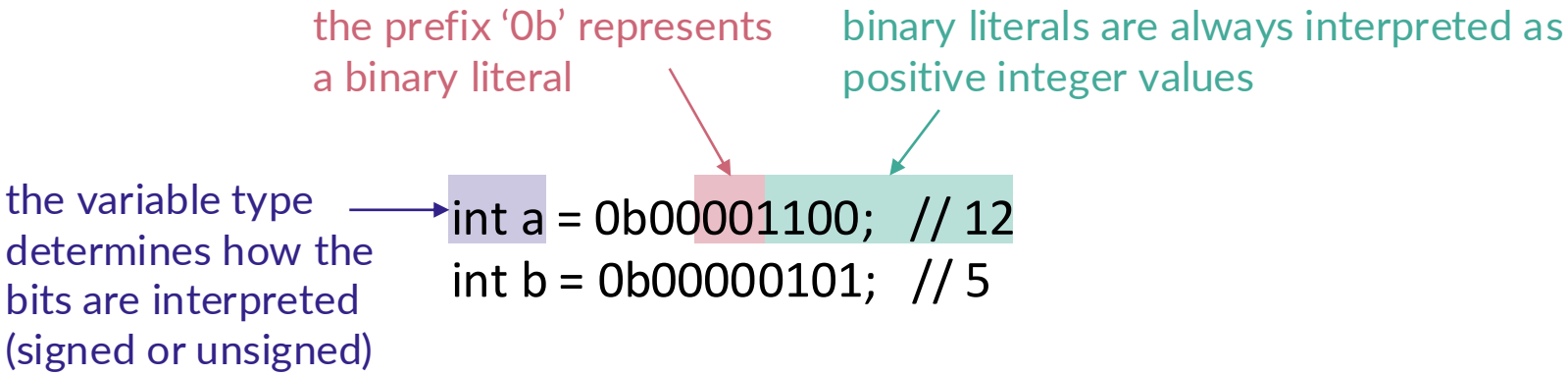
Bitwise and Shift Operators in C

C provides bitwise operators for AND, OR, XOR, and NOT.



Bitwise and Shift Operators in C

C provides bitwise operators for AND, OR, XOR, and NOT.



Bitwise and Shift Operators in C

C provides bitwise operators for AND, OR, XOR, and NOT.

the prefix '0b' represents a binary literal

binary literals are always interpreted as positive integer values

the variable type determines how the bits are interpreted (signed or unsigned)

```
int a = 0b00001100;   // 12
int b = 0b00000101;   // 5

int and = a & b;   // 0b00000100 = 4
int or  = a | b;   // 0b00001101 = 13
int xor = a ^ b;   // 0b00001001 = 9
int not = ~a;      // 0b11110011 = -13
int shl = a << 1;  // 0b00011000 = 24
int shr = a >> 1;  // 0b00000110  = 6
```

Using Bitwise and Shift Operators

Bitwise operations are typically used to :

- Set a certain bit in a variable
- Clear a certain bit in a variable
- Flip a certain bit (0 ⇒ 1 or 1 ⇒ 0) in a variable
- Test a certain bit (0 or 1) in a variable

Shift operations are typically used to:

- Create bitmaps to use with bitwise operations
- Multiply and divide binary numbers

Demo: set_bit.c

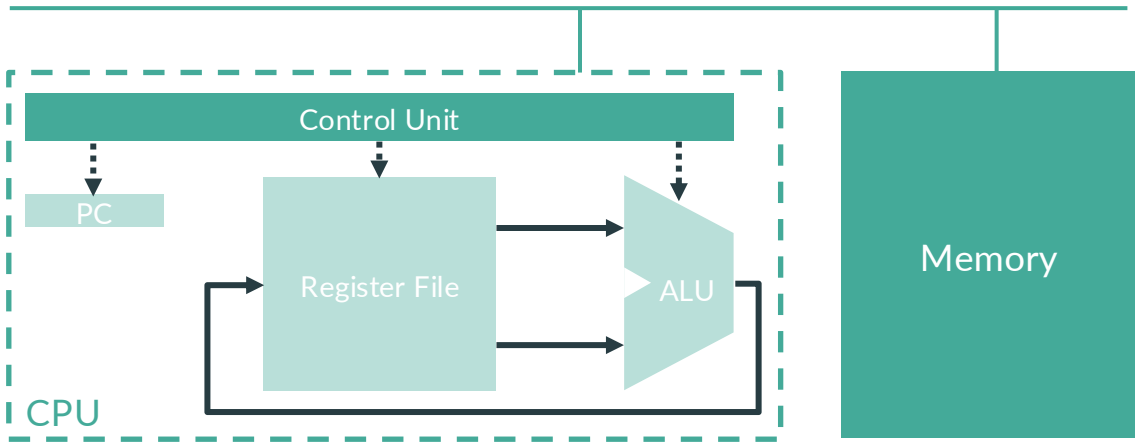
Exercise: clear_bit.c

Control Unit

The Control Unit Coordinates Execution

The *control unit* decodes the instruction and generates signals that:

- Select which registers provide input to the ALU
- Specify the operation performed by the ALU
- Enable memory reads and writes
- Drive the address bus for memory accesses
- Determine how the program counter (PC) is updated
- …

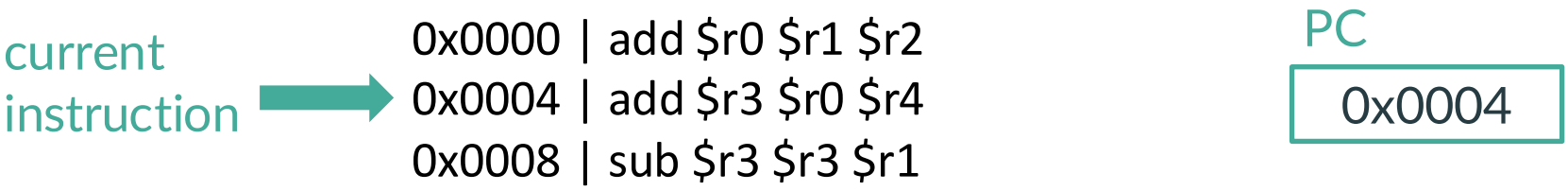


Program Counter (PC)

The PC is a Special Purpose Register

The *program counter (PC)* keeps track of where the CPU is in the program by storing the address of the instruction that's currently being executed.

At each new clock cycle, it updates to the address of the next instruction.



Where is the Next Instruction?

The next instruction is usually the next one in memory, unless the program jumps.

current instruction ➡ int z = 0;
next instruction ➡ if (x > y) {
    z = x – y;
} else {
    z = y – x;
}
z = z * 2;

Where is the Next Instruction?

The next instruction is usually the next one in memory, unless the program jumps.

current instruction
next instruction

```
int z = 0;
if (x > y) {
   z = x – y;
} else {
   z = y – x;
}
z = z * 2;
```

current instruction
next instruction

```
int z = 0;
if (x > y) {
   z = x – y;
} else {
   z = y – x;
}
z = z * 2;
```

CPU Structure

Data starts in the register file and goes through the ALU for computation. It is directed by the control unit, and sequenced by the program counter.

